



**Sunbeam Institute of Information Technology
Pune and Karad**

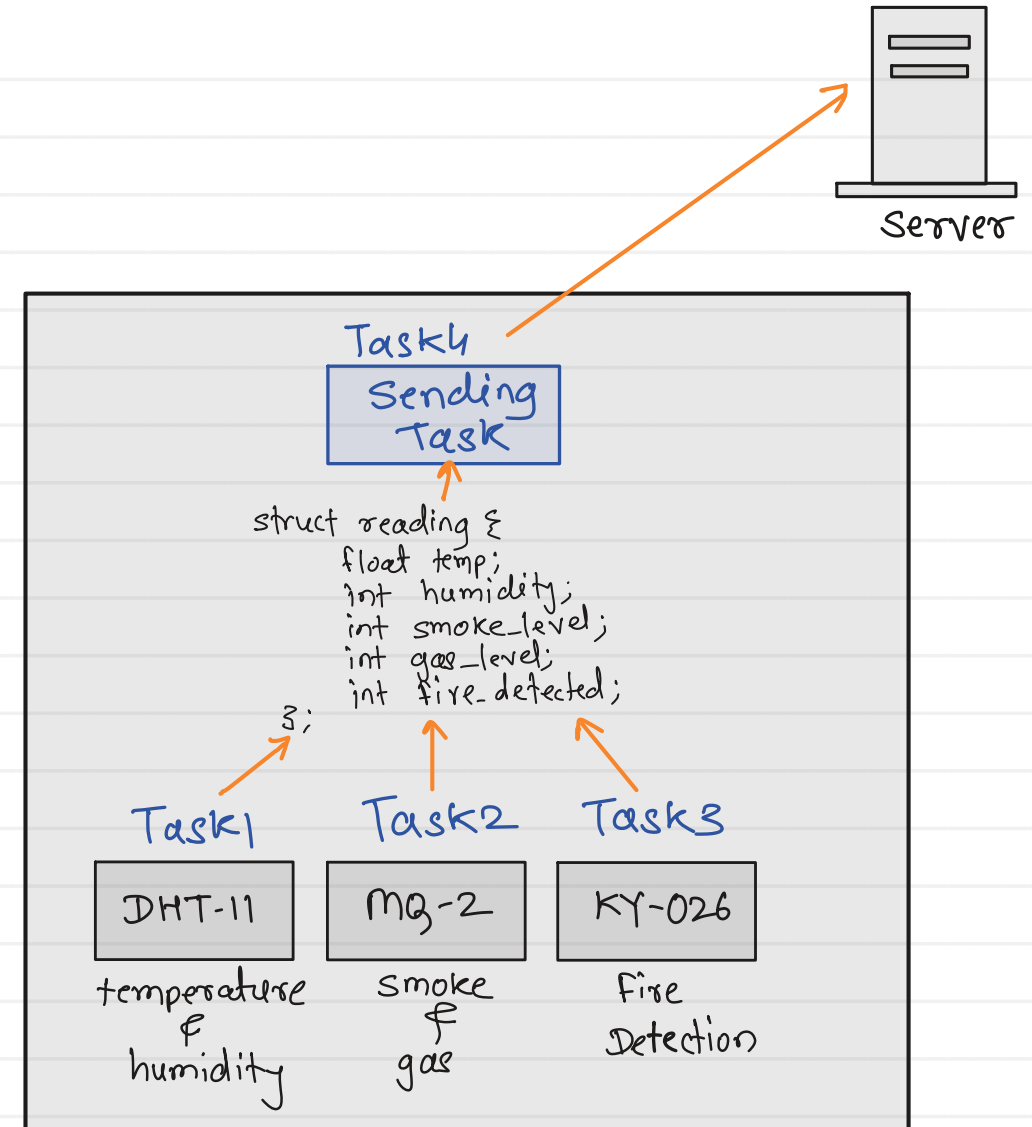
**Embedded and RTOS (FreeRTOS) Training
At RIT College, Islampur**

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Why to use FreeRTOS in Embedded ?

- ✓ Abstracting away timing information
- ✓ Maintainability/Extensibility
- ✓ Modularity
- ✓ Team development
- ✓ Easier testing
- ✓ Code reuse
- ✓ Improved efficiency
- ✓ Idle time
- ✓ Flexible interrupt handling
- ✓ Mixed processing requirements
- ✓ Easier control over peripherals



- Preemptive Multitasking
 - Highest priority ready task gets CPU.
- Priority-Based Scheduling
 - Supports multiple task priorities.
- Task Management
 - Create, delete, suspend, and resume tasks.
- Deterministic Behavior
 - Predictable execution and response times.
- Inter-Task Communication
 - ✓ • Queues ← ITC mechanism
 - ✓ • Message Buffers ← circular arrays
 - ✗ • Stream Buffers ← circular arrays used as a file
- Task Synchronization
 - ✓ • Semaphores
 - ✓ • Mutexes
 - ✓ • Event Groups
- ✓ • Software Timers
 - Execute functions after specified delays.
- ✓ • Memory Management
 - Multiple heap management schemes (heap_1 to heap_5). most popular → heap-4
- Low Memory Footprint
 - Suitable for resource-constrained microcontrollers. Flash - 6 Kb
RAM - few hundred bytes
- Tick and Tickless Operation
 - Supports low-power applications.
- Portable
 - Runs on a wide range of MCU architectures.
- Open Source
 - Distributed under the MIT License.

- **What is a Task?**
- Basic unit of execution in FreeRTOS
- Similar to a thread in traditional operating systems
- Each task has:
 - ✓ Task Control Block (TCB)
 - ✓ Stack
 - ✓ Priority
 - ✓ State
 - ✓ Task function

- **Types of task :**
 - Periodic Tasks
 - Aperiodic Tasks
 - Sporadic Tasks
 - High / Medium / Low Priority Tasks
 - Idle Task
 - Timer Service Task

- **Periodic Tasks**

- Execute at fixed time intervals
- Typically use vTaskDelay() or vTaskDelayUntil()
- Common in control systems

- **Aperiodic Tasks** *(one shot tasks)*

- Execute only when an event occurs
- No fixed execution interval
- Usually wait on queues, semaphores, or notifications

- **Sporadic Tasks**

- Event-driven tasks with a minimum separation time between activations
- Similar to aperiodic tasks but execution frequency is controlled

- **Examples:**

- Sensor Sampling
- LED Blinking
- Temperature Monitoring

- **Examples:**

- Button Press Handling
- UART Data Reception
- Network Packet Processing

- **Examples:**

- Alarm Processing
- Emergency Event Handling

- **High Priority Tasks**

- Time-critical operations
- Execute before lower-priority tasks

- **Examples:**

- Motor Control
- Safety Monitoring

- **Medium Priority Tasks**

- Important but not critical

- **Examples:**

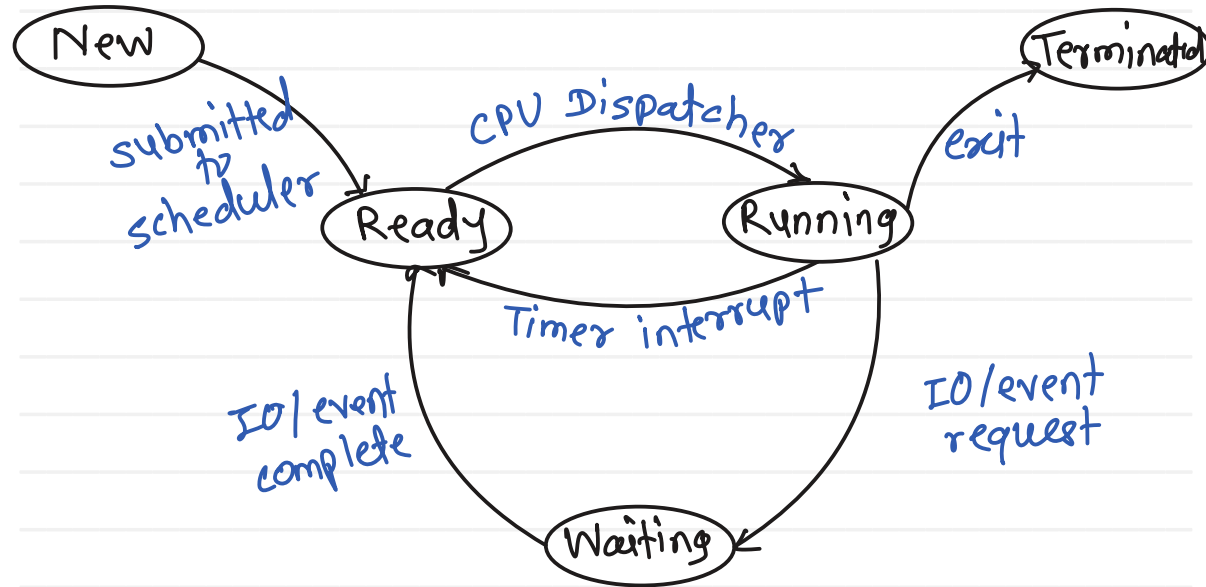
- Communication Tasks
- Data Processing

- **Low Priority Tasks**

- Background activities

- **Examples:**

- Logging
- Diagnostics
- Status Display



1. running → terminated } voluntarily
2. running → waiting } voluntarily
3. running → ready } forcefully
4. waiting → ready } forcefully

Types of scheduling :

1. Pre emptive :

- CPU access is given to next process forcefully.
- high priority process will not wait for low priority

2. Non Pre emptive :

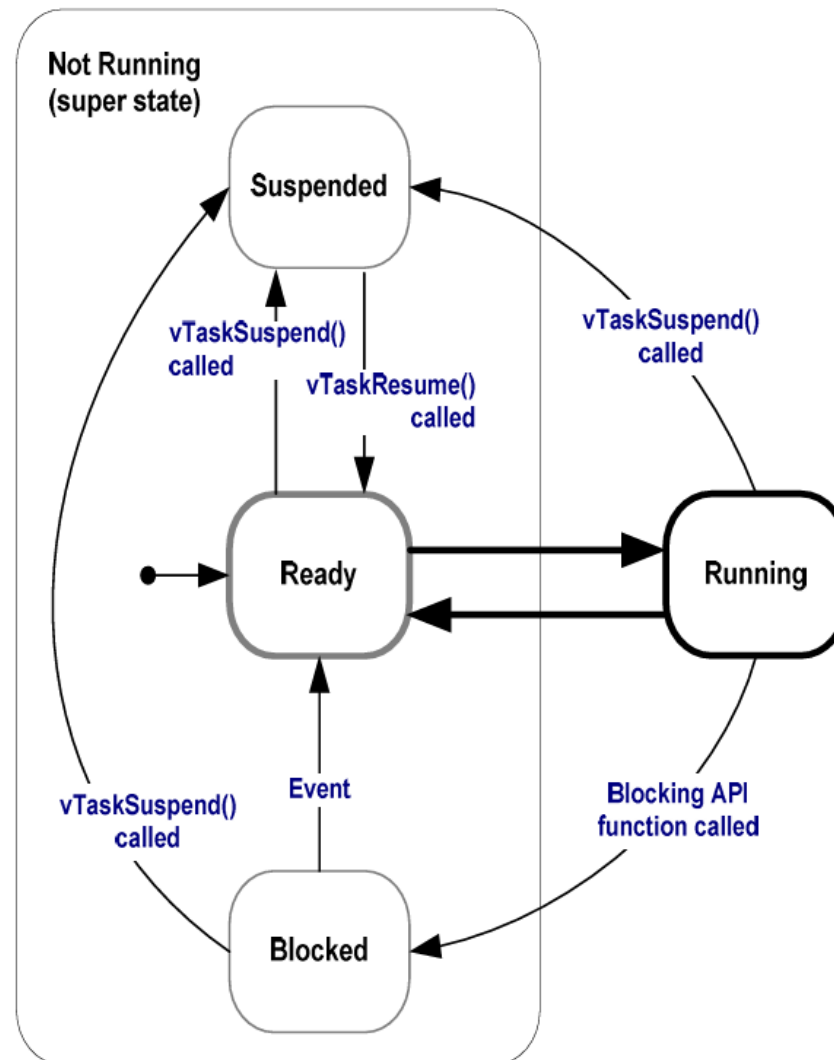
- CPU access is always given to next process voluntarily.
- cooperative scheduling
- high priority process may wait for low priority process.

- **FreeRTOS Scheduler**

- Core component of the FreeRTOS Kernel.
- Maintains task states and priorities.
- Performs context switching. (changing the context of CPU)
- Supports preemptive and cooperative scheduling.
- Ensures efficient CPU utilization.

- **Task States**

- Running
 - Currently executing on CPU.
- Ready
 - Ready to execute and waiting for CPU.
- Blocked
 - Waiting for an event or timeout.
- Suspended
 - Explicitly suspended by application.



- **Priority-Based Scheduling**

- Each task has a priority.
- Highest priority READY task runs first.
- Lower priority tasks run only when higher priority tasks are blocked.

- **Preemptive Scheduling**

- Default scheduling mode in FreeRTOS.
- Higher-priority task can interrupt a running lower-priority task.
- Provides fast response to critical events.

- **Round Robin Scheduling**

- Used among tasks of same priority.
- CPU time shared equally.
- Implemented using time slicing.

- **Cooperative Scheduling**

- Running task voluntarily yields CPU.
- No forced preemption.
- Lower scheduling overhead.
- API - taskYIELD();

give up

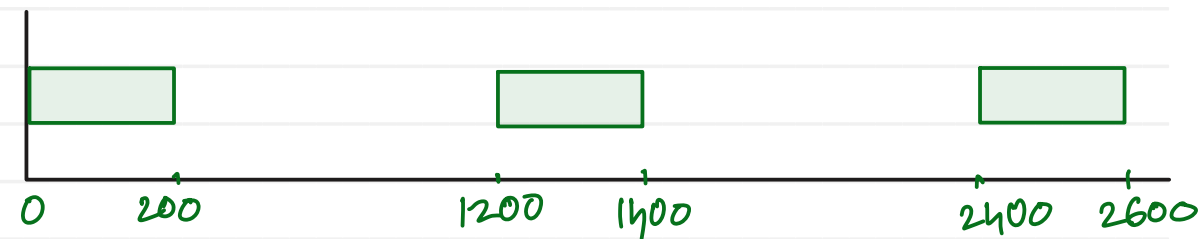
- **Time Slicing**

- Equal-priority tasks share CPU time.
- Controlled by system tick.
- Improves fairness.

```

vAperiodicTask( ) {
    while(1) {
        LED_Toggle(...); ← to do this need
        vTaskDelay(1000); 200 ticks
    }
}

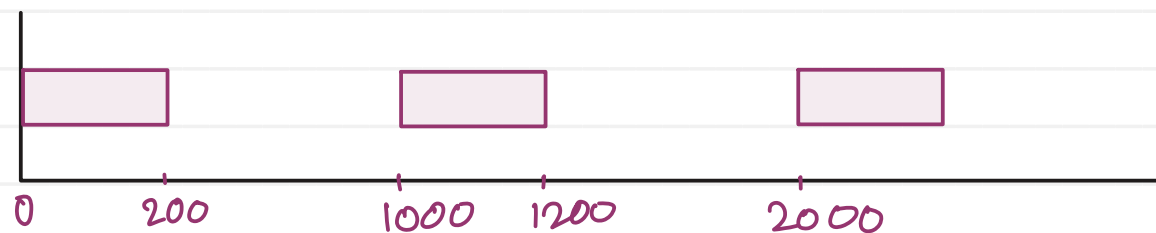
```



```

vPeriodicTask( ) {
    xLastWakeup = xTaskGetTickCount();
    while(1) {
        LED_Toggle(...);
        vTaskDelayUntil(&xLastWakeup, 1000);
    }
}

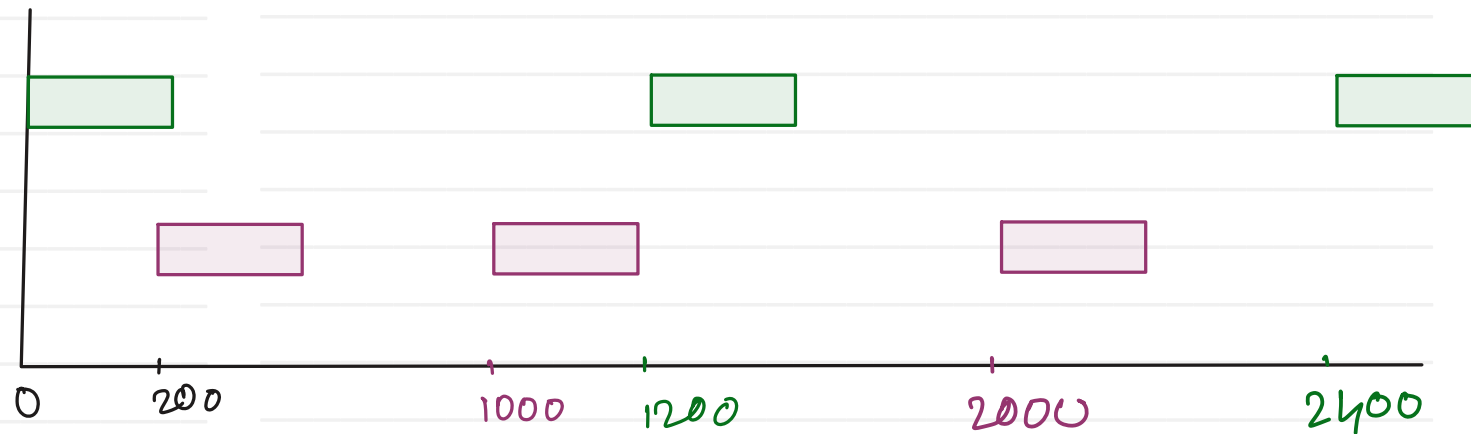
```



```

vAperiodicTask( ) {
    while(1) {
        LED_Toggle(...);
        vTaskDelay(1000);
    }
}
    
```

(h)



(3)

```

vPeriodicTask( ) {
    xLastWakeup = xTaskGetTickCount();
    while(1) {
        LED_Toggle(...);
        vTaskDelayUntil(&xLastWakeup, 1000);
    }
}
    
```

(static/fixed priority)

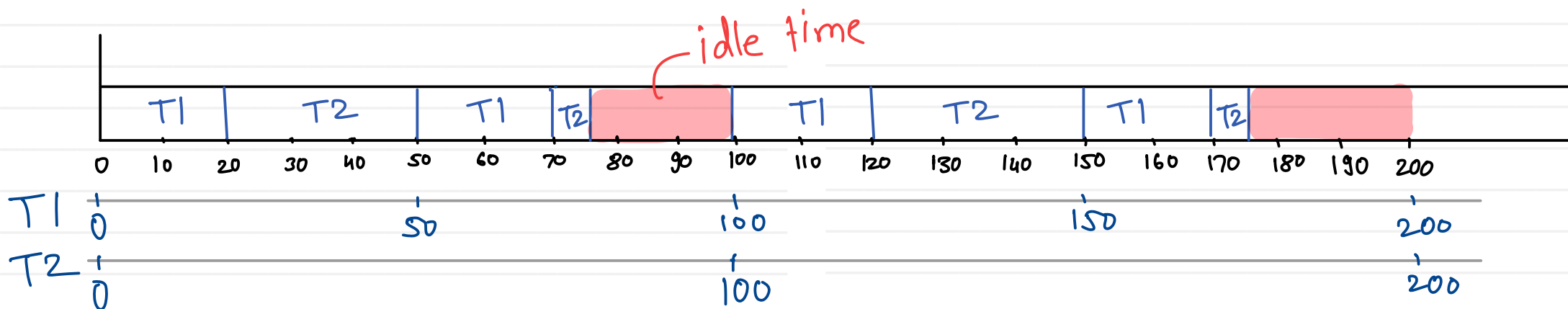
	CPU burst	Deadline	Period	Priority
T1	20	50	50	4 ← assume
T2	35	100	100	3

$$\text{Priority} \propto \frac{1}{\text{period}}$$

Priorities: $T1 > T2$

Hyper period: $\text{LCM}(50, 100) = 100$

Release point / phase: 0



$$\text{CPU Utilization} = \frac{75}{100} = 0.75$$

(static/fixed priority)

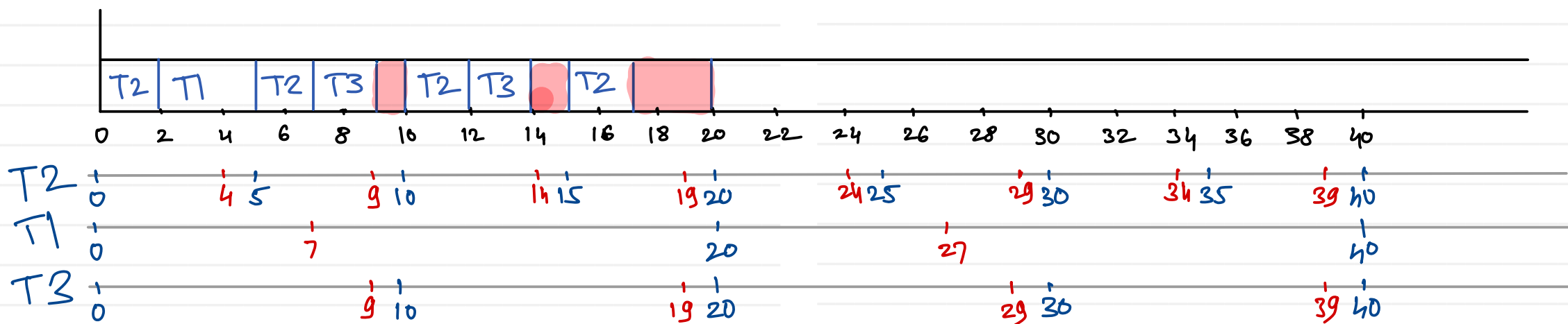
	CPU burst	Deadline	Period	Priority
T1	3	7	20	4
T2	2	4	5	5
T3	2	9	10	3

$$\text{Priority} \propto \frac{1}{\text{deadline}}$$

$$\text{Hyper period} = \text{LCM}(20, 5, 10) = 20$$

$$\text{Priorities} = T2 > T1 > T3$$

$$\text{Release point} = 0$$



$$\text{CPU Utilization} = \frac{15}{20} = 0.75$$

(dynamic/variable priority)

priority $\propto$ earliest deadline

Hyper period = LCM(50, 80) = 400

Release point = 0

